Volcano for the Agentic AI Era:

Unified Scheduling Across Training, Inference, and Agents

Kevin Wang · Ricardo Aravena · Chen Zicong

KubeCon + CloudNativeCon North America 2026
Salt Lake City · November 9–12

Different Workloads, Shared Infrastructure

The infrastructure must support different arrival patterns and coordination needs.

Different demands

Training & RL

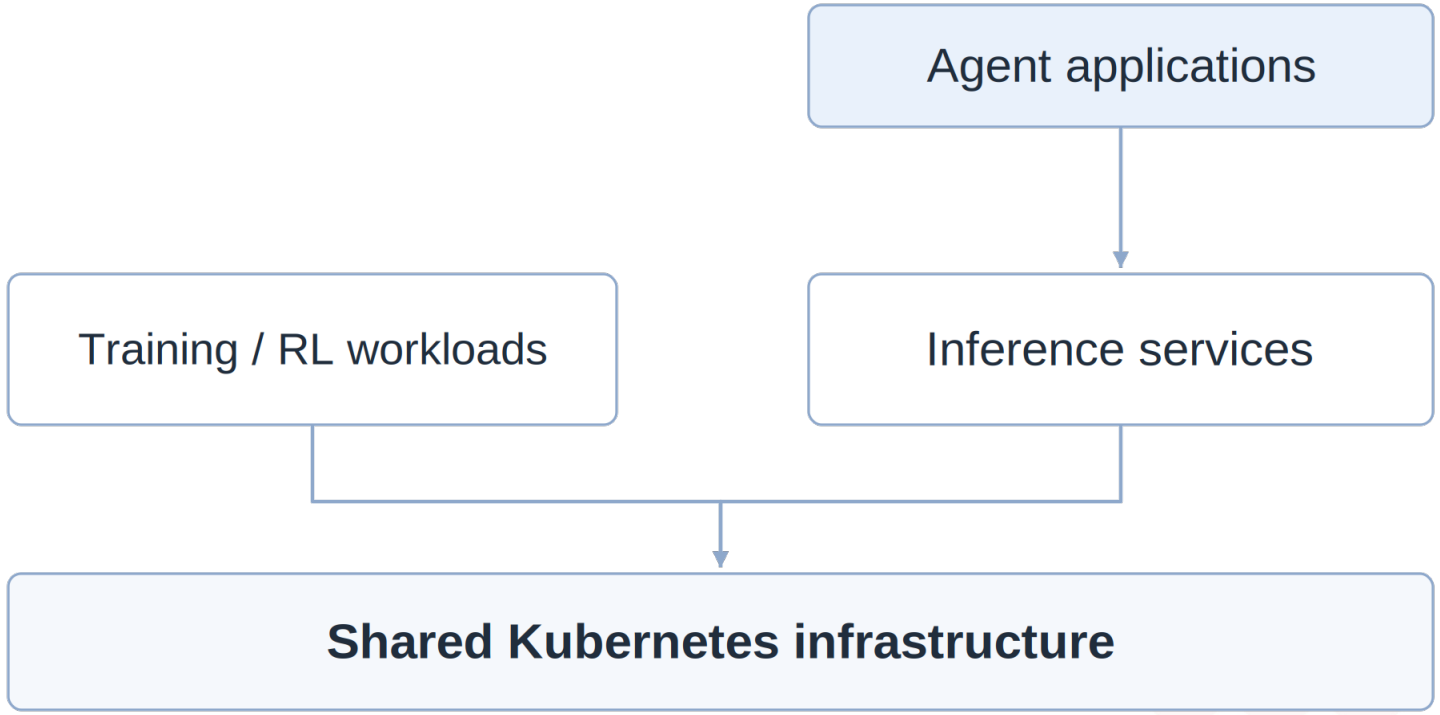
Coordinated, resource-intensive compute

Inference

Long-running services with distinct roles

Agent applications

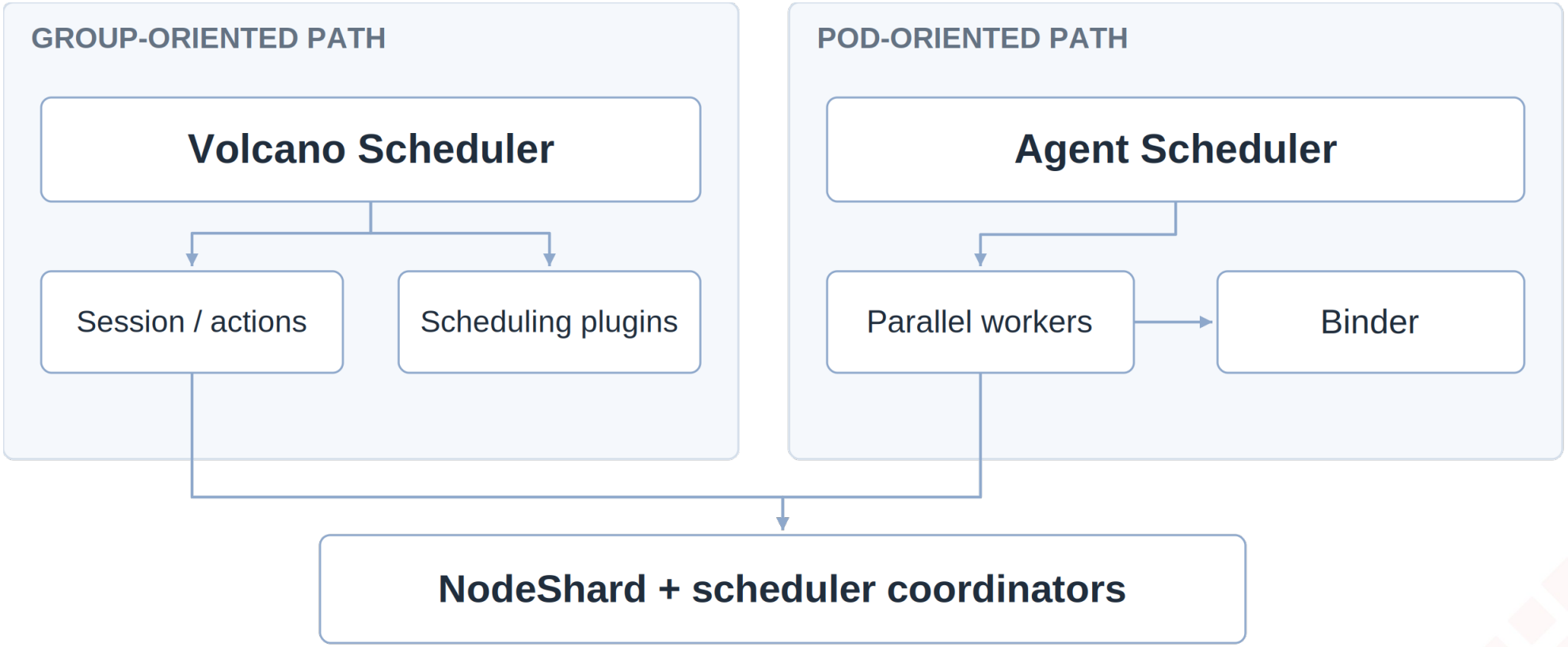
Bursty requests and latency-sensitive interactions



Different scheduling requirements, one shared cluster.

A Coordinated Scheduling Framework

Separate workload paths, coordinated node access, and local worker conflict handling.



Distinct paths

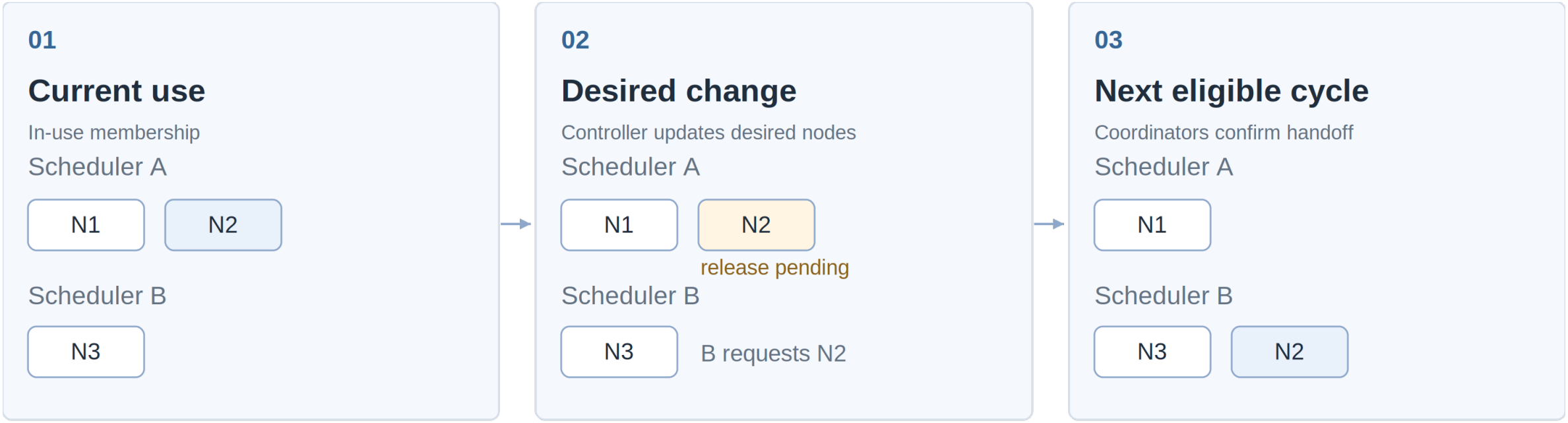
Workload-specific scheduling behavior

Local conflict handling

Coordinate candidate-node access across scheduling paths.

NodeShard: Coordinating Node Access

A shard change affects future placement; running Pods do not move with the shard.



NodeShard changes coordinate future placement.

none: no sharding soft: prefer shard nodes hard: restrict candidate nodes

Available Nodes Are Only the Beginning

Replica identity and topology matter alongside resource quantity.

Four Pods can still be the wrong four Pods.



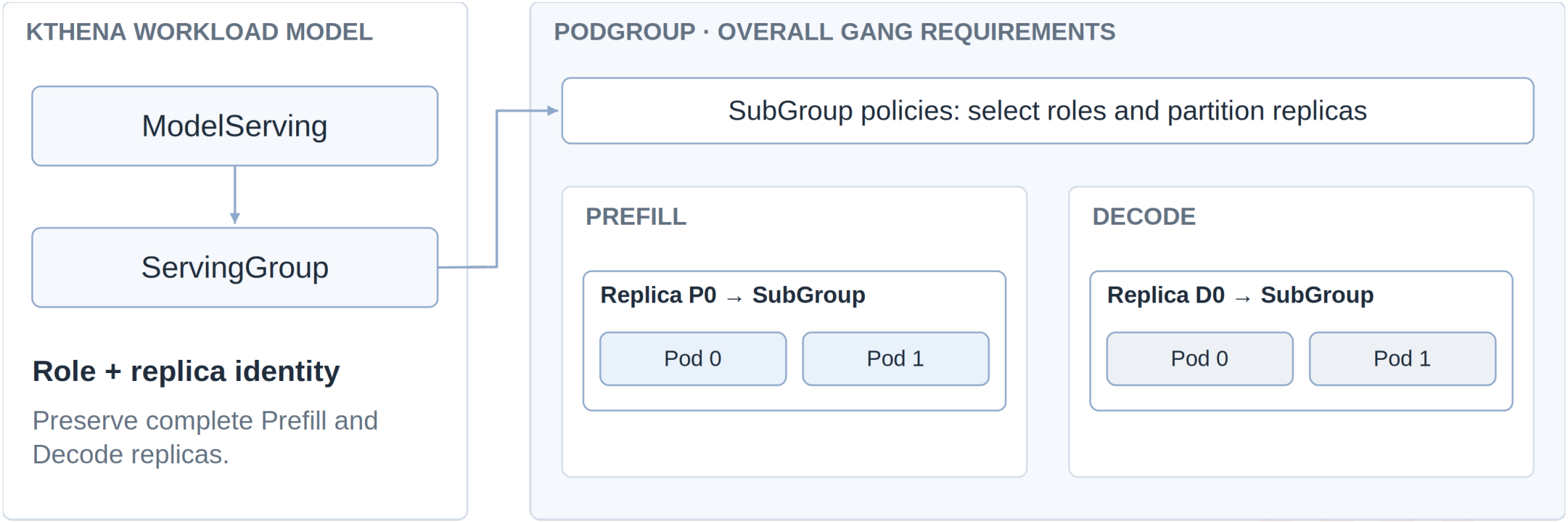
One required replica is incomplete.

Two complete two-Pod replicas.

Next: preserve group structure, then choose the right topology.

PodGroup and SubGroup: Complete Replicas

Use role and replica identity to describe what must be scheduled together.



One policy can generate multiple actual SubGroups.

HyperNode: Expressing Topology Domains

Topology boundaries express where a group can—or should—fit.

A hierarchy of domains

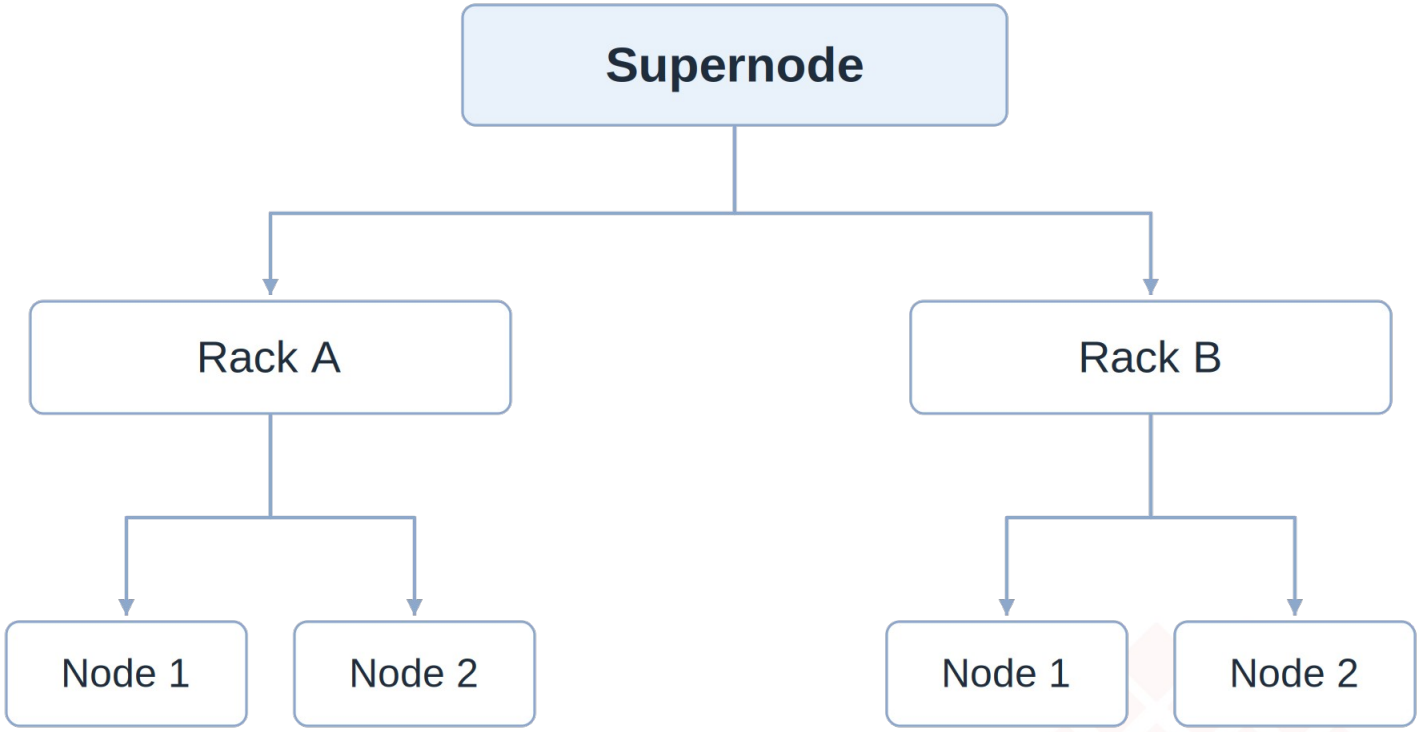
HyperNode models network topology across nodes.

Hard constraint

Require a valid topology domain.

Soft constraint

Prefer locality when feasible.



A topology domain describes locality, not scheduler ownership.

Place Groups at Different Topology Levels

Keep the service instance local while preserving each complete replica.

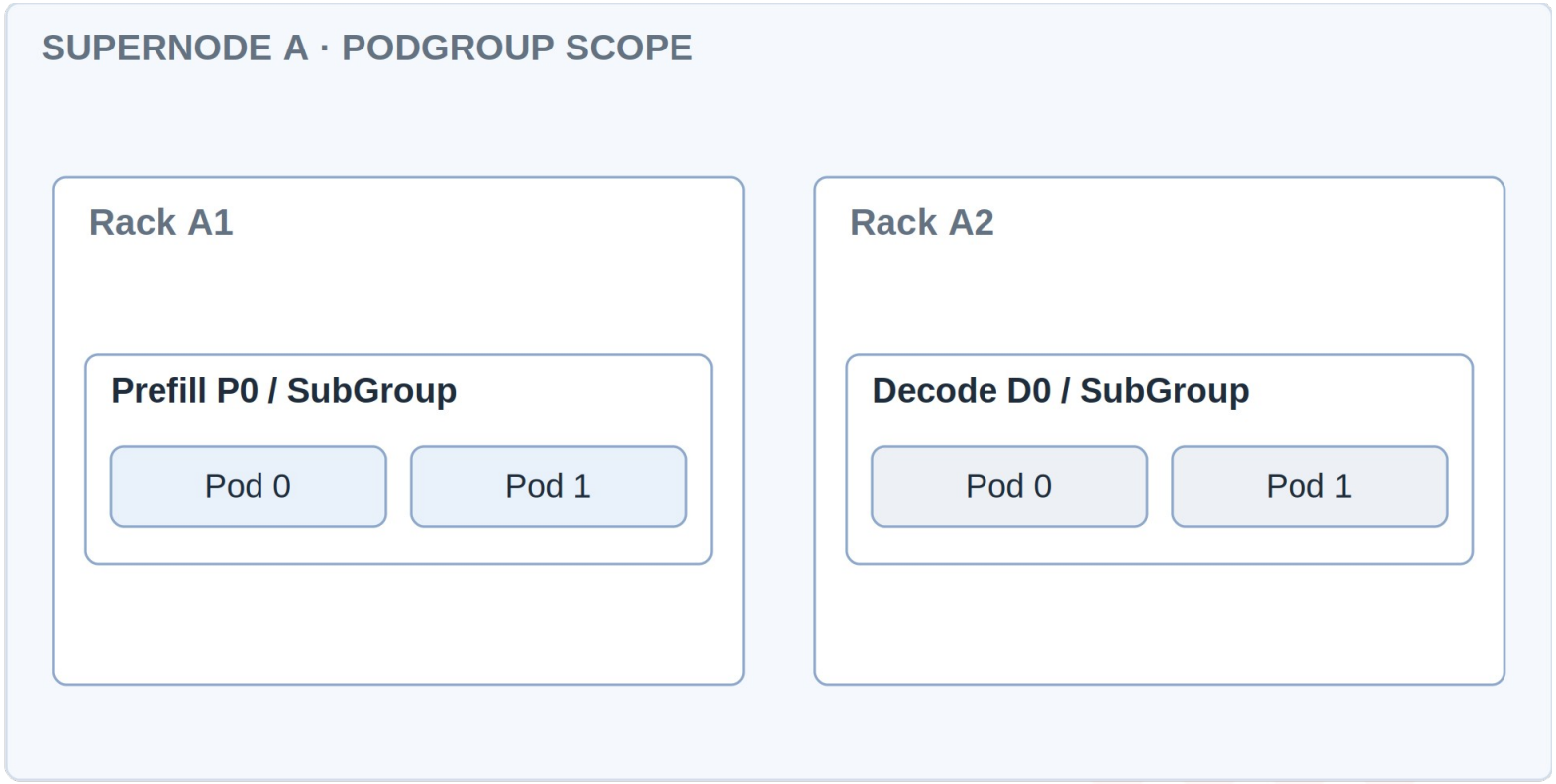
Two placement levels

PodGroup

Keep one serving instance inside a larger domain.

SubGroup

Keep each complete replica inside a smaller domain.

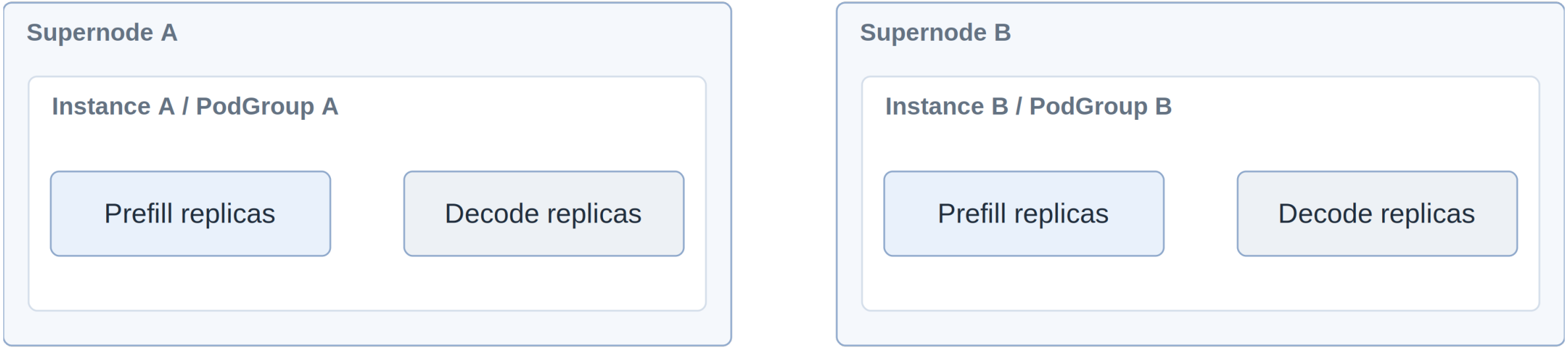


Capacity must fit within the selected topology domains.

Keep Each Instance Together, Place Instances Apart

Combine group-local topology with PodGroup anti-affinity.

AVAILABLE IN v1.16



Within each instance: network topology

Between instances: anti-affinity

Required: wait if no legal domain exists. Preferred: allow a feasible fallback.

Controllers Bring Workload Structure to Scheduling

Adoption and integration depth are different questions.

ECOSYSTEM	AVAILABLE / MERGED	EXPANDING
Kthena	Role / replica gang + topology	Reference scenarios for group relations
KubeRay	SubGroup support merged	Replica topology PR in progress
Grove	Gang / SubGroup backend merged	Topology mapping remains open
Kubeflow Trainer	Existing Volcano integration	Finer TrainJob groups + topology
LeaderWorkerSet	Existing gang / Volcano integration	Explore roles, slices, and group relations

Integration depth differs: gang support does not automatically include topology.

KubeRay: A Replica Can Span Multiple Pods

Two complete two-Pod replicas are more specific than a count of four.

One worker group

```
minReplicas: 2
numOfHosts: 2
```

Two complete replicas, each spanning two Pods.

Head group omitted here.

SUBGROUP POLICY PRESERVES LOGICAL REPLICA IDENTITY



GANG: MERGED

TOPOLOGY: IN PROGRESS

A WorkerGroup can produce multiple actual SubGroups.

Extending the Ecosystem and Placement Model

Reuse topology components and extend the workload semantics carried by controllers.

AVAILABLE

Standalone HyperNode

Discovery + controller
Reusable topology API

Scheduler adapters are separate integration work.

EXPANDING

Controller mappings

KubeRay / Grove
Kubeflow / LWS

Carry role, replica, and topology intent through.

PROPOSED

SubGroup relationships

Affinity / anti-affinity
Within one PodGroup

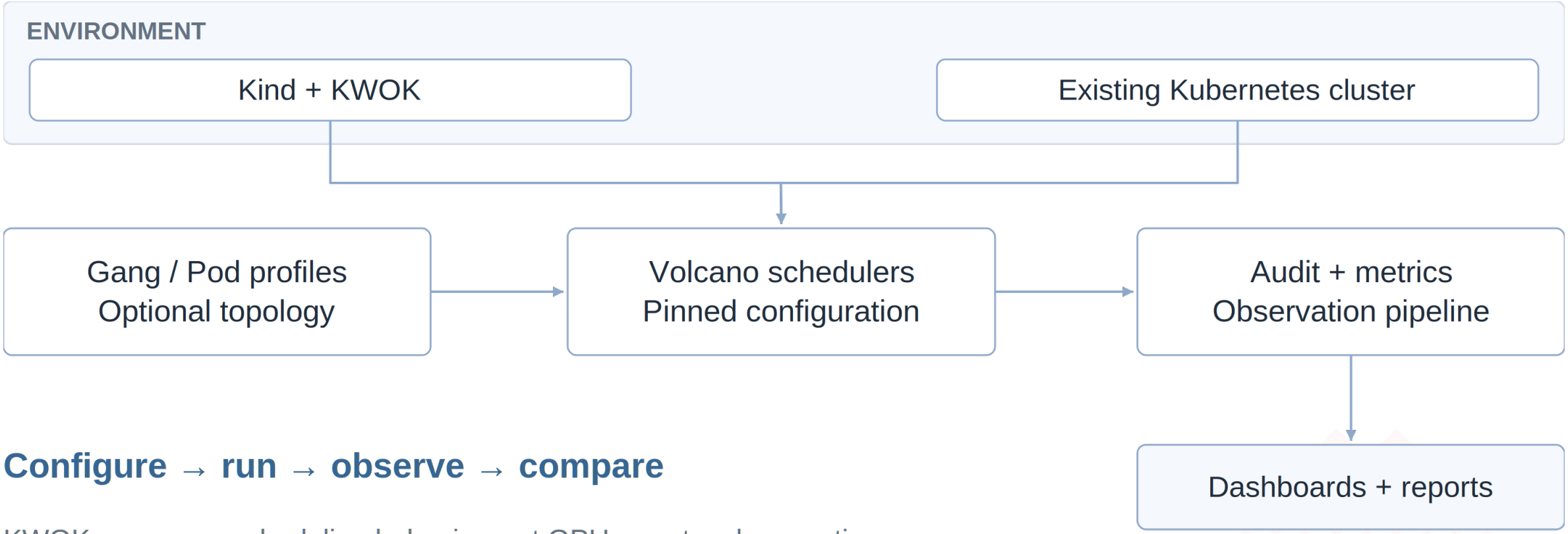


Express “near” or “apart”.

More workload shapes call for repeatable performance evaluation.

A Repeatable Scheduling Benchmark Framework

A common workflow for scenario setup, execution, observation, and comparison.



Configure → run → observe → compare

KWOK measures scheduling behavior, not GPU or network execution.

Benchmark Progress and Ongoing Validation

The framework is available; scenario coverage and performance validation continue.

FRAMEWORK IN PLACE

Workload profiles

Gang jobs, bare Pods, and topology configurations

Observability

Audit-based latency, scheduler metrics, and dashboards

Automation

Setup scripts, test execution, and report collection

ONGOING VALIDATION

Workload scale and shape

Vary job size, Pod count, and placement constraints

Optimization comparisons

Keep configuration and measurement windows comparable

Repeatability

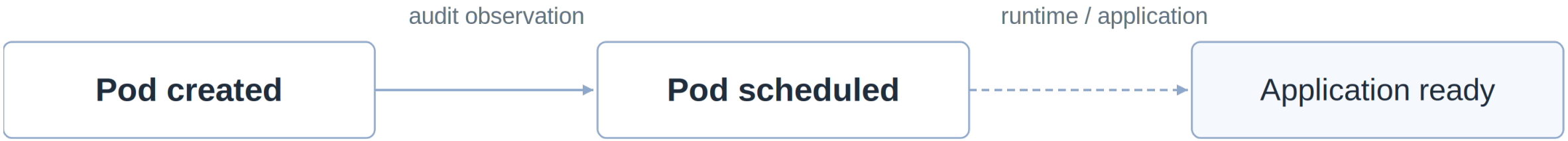
Re-run observations and check where bottlenecks move

Testing and optimization continue across workload scenarios.

From Measurements to Reliable Comparisons

Keep scheduling metrics distinct from end-to-end application readiness.

Measure the scheduling path before attributing the result.



CURRENT MEASUREMENT

Created → scheduled latency

Scheduling throughput and stage metrics

COMPARISON CONTEXT

Version · resources · workload · rules

Repeat runs; retain raw observations.

Current focus: reproducible comparisons and continued testing.

Framework, Workloads, Ecosystem, Evidence

Coordinate placement, carry workload intent, and validate behavior.

01 Unified scheduling

Different paths coordinate access to shared infrastructure.

02 Workload-aware placement

Complete groups, network locality, and instance separation.

03 Ecosystem integration

Controllers bring role and replica identity into scheduling.

04 Benchmark and improve

Repeatable measurements guide ongoing validation.

Questions & Discussion

Help shape the next set of scheduling and integration scenarios.

What does your workload need to keep together—or place apart?

Bring us your workload shapes, topology constraints,
and reproducible scheduling observations.

github.com/volcano-sh/volcano

Roadmap and feedback: issue #6166